

EXPERIMENT-2

Aim: To create and implement a basic Blockchain using Python and demonstrate transaction processing, block mining using Proof-of-Work, SHA-256 hashing, blockchain validation, and interaction with the blockchain through Flask APIs and Postman.

Theory:

Blockchain

A blockchain is a chain of linked blocks that stores data in a secure and tamper-evident way. In this experiment, each block holds an index, a timestamp, a list of transactions, a proof value and the hash of the previous block.

The `previous_hash` field links every block to the one before it. If an earlier block is altered, its hash changes and the link to the next block breaks, which makes tampering easy to detect.

Genesis Block

The first block of the chain is the Genesis Block. It is created automatically when the Blockchain class is initialized, with proof 1 and `previous_hash` 0.

Transactions

Transactions record the data exchanged between participants. In this implementation, every transaction consists of:

- Sender
- Receiver
- Amount

Pending transactions wait in a list and are added to a block when it is mined.

Proof-of-Work

Proof-of-Work is the mining step. The miner keeps trying proof values until the SHA-256 hash of ($\text{proof}^2 - \text{previous_proof}^2$) satisfies a fixed condition.

For this experiment, the hash must begin with three leading zeros:

000

SHA-256

SHA-256 is a cryptographic hash function that turns block data into a fixed 256-bit (64 hex character) digest. It is used in mining and block linking, and protects the integrity of the chain.

Blockchain Validation

A blockchain is valid when:

1. The `previous_hash` of each block matches the actual hash of its previous block.
2. The Proof-of-Work condition is satisfied.

Flask API

Flask is a lightweight Python framework used to expose the blockchain through HTTP APIs. Separate endpoints add transactions, mine blocks, return the chain, validate it and verify Proof-of-Work, and Postman is used to send requests and view the JSON responses.

CODE

Step 1 — Import Libraries

Colab Cell 1

```
import datetime

import hashlib

import json

print("Blockchain libraries loaded successfully!")
```

Step 2 — Install Flask

Colab Cell 2

```
!pip install flask -q

print("Flask installed successfully!")
```

Step 3 — Create Blockchain Class

```
class Blockchain:

    def __init__(self):

        self.chain = []

        self.pending_transactions = []

        self.add_block(

            proof=1,

            previous_hash="0",

            transactions=[]

        )

print("Blockchain class initialized!")

    def add_block(self, proof, previous_hash, transactions):
```

```

block = {
    "index": len(self.chain) + 1,
    "timestamp": str(datetime.datetime.now()),
    "transactions": transactions,
    "proof": proof,
    "previous_hash": previous_hash
}

```

```

self.chain.append(block)

```

```

return block

```

Step 4 — Implement Proof-of-Work

```

class Blockchain(Blockchain):
    def find_proof(self, previous_proof):
        proof = 1
        while True:
            operation = proof ** 2 - previous_proof ** 2
            hash_result = hashlib.sha256(
                str(operation).encode()
            ).hexdigest()
            if hash_result.startswith("000"):
                return proof
            proof += 1

```

Step 5 — Implement Block Hashing

```

class Blockchain(Blockchain):
    @staticmethod
    def calculate_hash(block):
        block_string = json.dumps(
            block,
            sort_keys=True
        ).encode()
        return hashlib.sha256(

```

```
        block_string
    ).hexdigest()
```

Step 6 — Implement Blockchain Validation

```
class Blockchain(Blockchain):
```

```
    def validate_chain(self):

        if len(self.chain) == 0:
            return False

        for position in range(1, len(self.chain)):

            previous = self.chain[position - 1]
            current = self.chain[position]

            if current["previous_hash"] != self.calculate_hash(previous):
                return False

            operation = (
                current["proof"] ** 2
                - previous["proof"] ** 2
            )

            hash_result = hashlib.sha256(
                str(operation).encode()
            ).hexdigest()

            if not hash_result.startswith("000"):
                return False

        return True
```

Step 7 — Create Blockchain Instance

```
blockchain = Blockchain()
```

```
print("Blockchain initialized successfully.")
print("Genesis Block:")
print(blockchain.chain[0])
```

```
*** Blockchain created successfully!
Genesis Block:
{'index': 1, 'timestamp': '2026-09-02 15:22:06.774129', 'proof': 1, 'previous_hash': '0', 'transactions': []}
```

Step 8 — Add a Transaction

```
next_block = blockchain.add_transaction(
    "Gitesh",
    "Jatin",
    100
)

print("Transaction added successfully.")
```

-
- Transaction added successfully!
[{'sender': 'Gitesh', 'receiver': 'Jatin', 'amount': 100}]

Step 9 — Mine a Block

```
def mine_block():
    # Check transaction list
    if len(blockchain.transactions) == 0:
        return {
            'message': 'No transactions to mine'
        }
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    # Proof of Work
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    # Take transactions
    transactions = blockchain.transactions.copy()
    # Remove transactions from pending list
    blockchain.transactions = []
    # Create block
    block = blockchain.create_block(
        proof,
        previous_hash,
        transactions
    )
    return {
        'message': 'Block mined successfully!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }
```

Step 10 — Display the Blockchain

```
print("Complete Blockchain:")
print(blockchain.chain)
```

Complete Blockchain:

```
[{'index': 1, 'timestamp': '2026-09-02 15:22:06.774129', 'proof': 1, 'previous_hash': '0', 'transactions': []},  
{ 'index': 2, 'timestamp': '2026-09-02 15:22:06.811234', 'proof': 533, 'previous_hash':  
'392bfd07bd22bf7b8a3733701b11536a8dbbd629bd7347475b87d8c58136c423', 'transactions':  
[{'sender': 'Gitesh', 'receiver': 'Jatin', 'amount': 100}]}]
```

Step 11 — Validate the Blockchain

Checking if the Blockchain is valid

```
def is_valid():  
    is_valid = blockchain.is_chain_valid(blockchain.chain)  
    if is_valid:  
        response = {'message': 'All good. The Blockchain is valid.'}  
    else:  
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}  
    return response
```

Step 12 — Transaction Node

```
app_transaction = Flask("Transaction_Node")  
@app_transaction.route(  
    "/add_transaction",  
    methods=["POST"]  
)  
def transaction_api():  
    data = request.get_json()  
    required_fields = [  
        "sender",  
        "receiver",  
        "amount"  
    ]  
    if not all(  
        field in data  
        for field in required_fields  
    ):  
        return jsonify(  
            "message": "Missing transaction details"  
        ), 400  
    next_block = blockchain.add_transaction(  
        data["sender"],  
        data["receiver"],  
        data["amount"]  
    )
```

```

return jsonify({
    "status": "success",
    "message": "Transaction added successfully",
    "next_block": next_block
}), 201

```

Step 13 — Mining Node

```

app_mining = Flask("Mining_Node")

@app_mining.route(
    "/mine_block",
    methods=["GET"]
)
def mining_api():

    result = perform_mining()

    if result["message"] == "No transactions to mine":
        return jsonify(result), 400

    return jsonify(result), 200

```

Step 14 — Blockchain Node

```

app_chain = Flask("Blockchain_Node")

@app_chain.route(
    "/get_chain",
    methods=["GET"]
)
def blockchain_api():

    return jsonify({
        "length": len(blockchain.chain),
        "chain": blockchain.chain
    }), 200

```

Step 15 — Validation Node

```

app_validity = Flask("Validation_Node")

@app_validity.route(
    "/is_valid",
    methods=["GET"]
)
def validation_api():

    status = blockchain.validate_chain()

```

```

if status:
    message = "Blockchain is valid"
else:
    message = "Blockchain is not valid"

return jsonify({
    "message": message
}), 200

```

Step 16 — Proof Verification Node

```

app_proof = Flask("Proof_Node")

@app_proof.route(
    "/check_proof",
    methods=["GET"]
)
def proof_api():

    previous = blockchain.latest_block()

    previous_proof = previous["proof"]

    proof = blockchain.find_proof(
        previous_proof
    )

    operation = (
        proof ** 2
        - previous_proof ** 2
    )

    proof_hash = hashlib.sha256(
        str(operation).encode()
    ).hexdigest()

    return jsonify({
        "previous_proof": previous_proof,
        "proof": proof,
        "hash": proof_hash,
        "valid": proof_hash.startswith("000")
    }), 200

```

Step 17 — Start All Nodes

```

def run_transaction():
    app_transaction.run(
        host="0.0.0.0",
        port=5000,

```

```
        debug=False,  
        use_reloader=False  
    )
```

```
def run_mining():  
    app_mining.run(  
        host="0.0.0.0",  
        port=5001,  
        debug=False,  
        use_reloader=False  
    )
```

```
def run_chain():  
    app_chain.run(  
        host="0.0.0.0",  
        port=5002,  
        debug=False,  
        use_reloader=False  
    )
```

```
def run_validity():  
    app_validity.run(  
        host="0.0.0.0",  
        port=5003,  
        debug=False,  
        use_reloader=False  
    )
```

```
def run_proof():  
    app_proof.run(  
        host="0.0.0.0",  
        port=5004,  
        debug=False,  
        use_reloader=False  
    )
```

```
threading.Thread(  
    target=run_transaction,  
    daemon=True  
)start()
```

```
threading.Thread(  
    target=run_mining,  
    daemon=True  
)start()
```

```
threading.Thread(  
    target=run_chain,  
    daemon=True  
)start()
```

```
threading.Thread(  
    target=run_validity,  
    daemon=True  
)start()  
  
threading.Thread(  
    target=run_proof,  
    daemon=True  
)start()  
  
print("All blockchain services started successfully!")
```

Step 18 — Generate Public URLs

```
from google.colab.output import eval_js
```

```
url5000 = eval_js(  
    "google.colab.kernel.proxyPort(5000)"  
)
```

```
url5001 = eval_js(  
    "google.colab.kernel.proxyPort(5001)"  
)
```

```
url5002 = eval_js(  
    "google.colab.kernel.proxyPort(5002)"  
)
```

```
url5003 = eval_js(  
    "google.colab.kernel.proxyPort(5003)"  
)
```

```
url5004 = eval_js(  
    "google.colab.kernel.proxyPort(5004)"  
)
```

```
print("Transaction Node:", url5000)  
print("Mining Node:", url5001)  
print("Blockchain Node:", url5002)  
print("Validation Node:", url5003)  
print("Proof Node:", url5004)
```

Output:

1. Flask Server Running:

```
C:\Users\INFT505-11\Desktop>python exp2bc.py
* Serving Flask app 'exp2bc'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.47.107:5000
Press CTRL+C to quit
```

2. Add Transaction:

The screenshot displays a REST client interface with the following details:

- URL:** `http://127.0.0.1:5000/add_transaction`
- Method:** `POST`
- Body (Request):**

```
{
  "sender": "Dipesh",
  "receiver": "Mayur",
  "amount": 10000000
}
```
- Status:** `201 CREATED`, Time: 4 ms, Size: 288 B
- Body (Response):**

```
{
  "amount": 10000000,
  "block_index": 3,
  "message": "Transaction added successfully.",
  "receiver": "Mayur",
  "sender": "Dipesh"
}
```

3. Mine Block:

HTTP http://127.0.0.1:5000/mine_block Save

GET http://127.0.0.1:5000/mine_block Send

Params Authorization Headers (6) **Body** Pre-request Script Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON** Beautify

1

Body Cookies Headers (5) Test Results Status: 200 OK Time: 15 ms Size: 443 B Save Response

Pretty Raw Preview Visualize **JSON** 🔍

```
1  {
2    "index": 3,
3    "message": "Congratulations, you just mined a block!",
4    "previous_hash": "171235d237e9ae267c55356139fedbdd75ce01b9a856c08ae83a47c2365520c0",
5    "proof": 11028,
6    "timestamp": "2026-09-02 12:04:34.124750",
7    "transactions": [
8      {
9        "amount": 10000000,
10       "receiver": "Mayur",
11       "sender": "Dipesh"
12     }
13   ]
14 }
```

4. Display Blockchain:

HTTP http://127.0.0.1:5000/get_chain Save

GET http://127.0.0.1:5000/get_chain Send

Params Authorization Headers (6) **Body** Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (5) Test Results Status: 200 OK Time: 4 ms Size: 737 B Save Response

Pretty Raw Preview Visualize **JSON** 🔍

```
1  {
2    "chain": [
3      {
4        "index": 1,
5        "previous_hash": "0",
6        "proof": 1,
7        "timestamp": "2026-09-02 11:56:43.014734",
8        "transactions": []
9      }
10    ]
11  }
```

```
Body Cookies Headers (5) Test Results
Status: 200 OK Time: 4 ms Size: 737 B Save Response
Pretty Raw Preview Visualize JSON
{
  "index": 2,
  "previous_hash": "fe53fb38a16dafa6edf17f44490fdd042efc721bb823bcf814306da23d32773f",
  "proof": 533,
  "timestamp": "2026-09-02 12:01:56.144796",
  "transactions": [
    {
      "amount": 10000000,
      "receiver": "Chirag",
      "sender": "Dipesh"
    }
  ]
}
```

```
Body Cookies Headers (5) Test Results
Status: 200 OK Time: 4 ms Size: 737 B Save Response
Pretty Raw Preview Visualize JSON
{
  "index": 3,
  "previous_hash": "171235d237e9ae267c55356139fedbdd75ce01b9a856c08ae83a47c2365520c0",
  "proof": 11028,
  "timestamp": "2026-09-02 12:04:34.124750",
  "transactions": [
    {
      "amount": 10000000,
      "receiver": "Mayur",
      "sender": "Dipesh"
    }
  ]
},
"length": 3
}
```

5. Blockchain Validation:

URL: http://127.0.0.1:5000/is_valid Save

Method: GET URL: Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

```
Body Cookies Headers (5) Test Results
Status: 200 OK Time: 3 ms Size: 214 B Save Response
Pretty Raw Preview Visualize JSON
{
  "message": "All good. The Blockchain is valid."
}
```

6. Proof - of - Work Verification

HTTP http://127.0.0.1:5000/check_proof Save

GET http://127.0.0.1:5000/check_proof Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (5) Test Results Status: 200 OK Time: 4 ms Size: 355 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "condition_satisfied": true,
3   "golden_nonce": 533,
4   "hash": "0000c00870f23a23ae80377298491b091db400d575be0efbde5b310f2f763ed1",
5   "previous_proof": 1,
6   "required_condition": "Hash must start with 000"
7 }
```

7. Proof API Request in Flask

```
C:\Users\INFT505-11\Desktop>python exp2bc.py
* Serving Flask app 'exp2bc'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.47.107:5000
Press CTRL+C to quit
127.0.0.1 - - [02/Sep/2026 12:09:31] "GET /check_proof HTTP/1.1" 200 -
127.0.0.1 - - [02/Sep/2026 12:09:41] "GET /check_proof HTTP/1.1" 200 -
```

Conclusion:

The basic concept of Blockchain was successfully implemented using Python. The experiment demonstrated transaction creation, block mining using Proof-of-Work, SHA-256 hashing, blockchain validation, and Flask-based API interaction. The blockchain was successfully verified as valid, demonstrating the basic principles of **immutability, hashing, and Proof-of-Work**.